

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

Домашнее задание 5

Выполнил:

Баженов Алексей

Группа К3340

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить RabbitMQ, реализовать асинхронное взаимодействие между микросервисами через очереди сообщений

Ход работы

1. Выбор брокера

Выбран RabbitMQ по причинам, в сравнении с Kafka:

- Легче настраивается
- Меньше потребление ресурсов

2. Реализованные события

Событие	Отправитель	Получатель	Назначение
review.created	review-service	restaurant-service	Асинхронное обновление рейтинга ресторана
reservation.created	reservation-service	restaurant-service	Уведомление о создании брони
reservation.cancelled	reservation-service	restaurant-service	Освобождение столика при отмене

3. Добавление RabbitMQ в docker-compose

Добавили в конфигурации docker-compose настройки RabbitMQ

4. Общий модуль для работы с RabbitMQ

Создан файл rabbitmq.ts для каждого сервиса-участника с функциями:

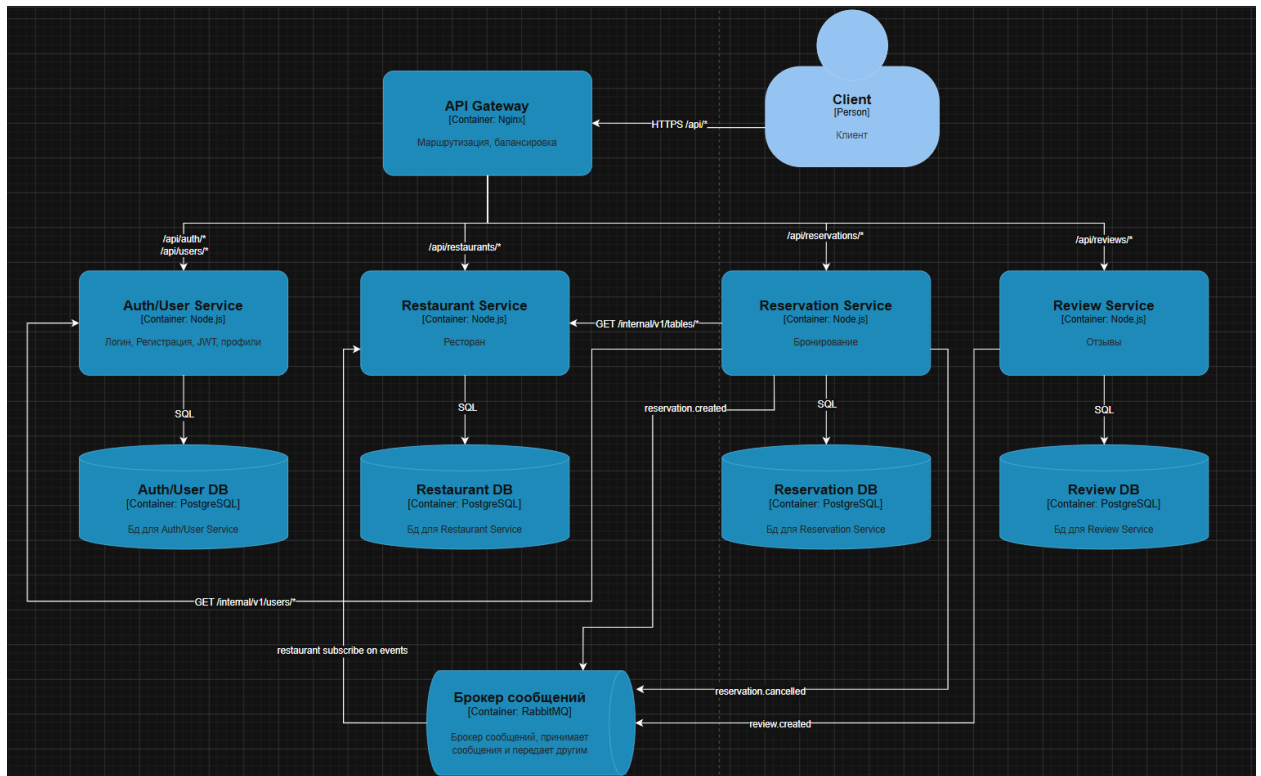
- getRabbitChannel() — подключение с retry-логикой
- publishEvent() — отправка события
- subscribeToEvents() — подписка на события

5. Изменения в сервисах

review-service — при создании отзыва отправляет review.created:

reservation-service — при создании и отмене брони отправляет события

restaurant-service — подписывается на все события и обрабатывает их



Результат

1. RabbitMQ развёрнут в Docker Compose
2. Доступен Management UI
3. Реализованы асинхронные события
4. Добавлена retry-логика для устойчивости к задержкам запуска
5. Сервисы общаются через брокер

Вывод

RabbitMQ успешно интегрирован в архитектуру микросервисов. Асинхронные события позволяют развязать сервисы и повысить отказоустойчивость системы. Клиент получает ответ мгновенно, не дожидаясь выполнения фоновых операций.